AegisBallot

Secure Local Electronic Voting Platform

Product Documentation • Version 1.0.0 • Local Demonstration

AegisBallot demonstrates an end-to-end encrypted, blockchain-backed voting workflow with synthetic identity verification, quorum-validated ledger, and independent auditability — all running locally on SQLite and a permissioned ledger simulation.

Feature	Implementation
Identity	DemoAadhaarProvider (Verhoeff, salted hash, masked display)
Ledger	3 validators, quorum 2/3, SHA-256, Ed25519
Ballot	AES-256-GCM, identity/ballot separation
Auth	Argon2id, server-side sessions, RBAC
DB	SQLite + SQLAlchemy 2.x

This is a functional local platform, not a production national election system. Demo verification does not contact UIDAI.

Product Overview

AegisBallot enables an administrator to create and configure elections, manage candidates, and open/close voting; a voter to authenticate, verify via synthetic Aadhaar, view candidates, confirm, cast exactly one encrypted ballot, and receive a non-revealing receipt; and an auditor to inspect the ledger, validate chain integrity, and run tamper detection.

Core principles: reliability over complexity, honest local simulation over exaggerated claims, and clear separation of identity from ballot.

Problem Addressed

Electronic voting requires trustworthy identity, one-vote enforcement, ballot secrecy, and verifiable integrity without relying on public blockchain infrastructure or Internet connectivity. AegisBallot shows how these properties can be realized locally with standard cryptography and permissioned ledger concepts.

Key Features & User Roles

Roles: ADMIN (election/candidate management, open/pause/close, results, system health), VOTER (login, Aadhaar verification, candidate browsing, ballot confirmation, receipt), AUDITOR (chain explorer, block/tx details, integrity verification, audit log, tamper demo).

Workflow: Admin creates election → synthetic voters registered → voter logs in → Demo Aadhaar verification → eligibility check → one-time authorization → candidate view → confirmation → encrypted ballot → quorum-validated block commit → receipt → duplicate-vote rejection → admin closes election → decrypted tally → auditor verification.

Architecture Summary

Four layers: (1) Persistent storage — SQLite with SQLAlchemy, foreign keys, unique constraints. (2) Ledger — local permissioned chain: genesis block, SHA-256, deterministic serialization, Merkle root, Ed25519 signatures, 3 validators. (3) Application/business logic — FastAPI services for elections, voting, identity, audit. (4) Frontend — Jinja2 templates, vanilla JS, light theme, responsive cards.

Modular paths: app/identity/provider.py exposes IdentityVerificationProvider; app/ledger/ledger.py exposes propose_block/validate_chain; app/security/crypto.py centralizes SHA-256, AES-GCM, Ed25519.

Identity-Verification Design

Synthetic 12-digit numbers are generated with Verhoeff check digits; validation checks length, digits, Verhoeff, existence in voter_profiles (hashed with salt+pepper), eligibility, and not-already-voted. Raw identifiers are not stored beyond a salted hash (SHA-256 of salt+pepper+number); UI shows masked form XXXX-XXXX-1234. The adapter is DemoAadhaarProvider; the code marks it as simulated/demo and documents that it can be swapped for a real provider without changing voting logic.

Blockchain / Ledger Design

Each block contains index, timestamp, previous_hash, merkle_root, block_hash, validator_signatures, transaction count. Hashing uses SHA-256 over canonical JSON serialization. Validators maintain independent keypairs (Ed25519) in data/validators.json. Normal flow: app builds transaction (ballot_id, election_id, encrypted_payload, nonce, timestamp, tx_hash, signature) → primary proposes block → replicas validate schema, previous hash, signature, nonce/tx_hash uniqueness → at least 2/3 approve → block committed → replicas sync → receipt returned. Immutability and tamper detection derive from hash chaining and recomputation.

Ballot Privacy Model

Separation: voter_profiles / identity_credentials hold identity; VotingAuthorization holds hashed one-time token; Ballot and LedgerTransaction hold only anonymous ballot_id, encrypted choice, nonce, tx_hash. Token is random (secrets.token_urlsafe), single-use, 10-minute expiry, hashed with SHA-256 at rest. Payload is encrypted via AES-256-GCM with a per-deployment key in data/secrets.json (never in source control). The receipt proves acceptance (election, ballot_id, tx_hash, block index/hash) without revealing candidate.

Security Controls

Argon2id password hashing; server-side sessions (httponly, lax samesite); CSRF via per-session token (X-CSRF-Token header check); rate limiting on login/verification (in-memory window); input validation (Pydantic where applicable); secure random IDs; SHA-256, AES-GCM, Ed25519; RBAC with server-side checks; DB constraints (uq_one_vote_per_voter, uq_auth_one_per_election); structured audit logs (no secrets); safe error messages.

Election Administration

Admin UI: create election (title, description, start/end, eligibility), add/edit/deactivate candidates (candidate_id, name, party, description, manifesto, active flag), view voter count, open/pause/close, view results after closure, view benchmark summary, ledger height, validator status, last hash, chain integrity, recent audit events. All metrics derive from DB/ledger/benchmarks.

Auditor Capabilities

Auditor dashboard provides chain explorer (blocks, tx), block/tx details, validator approvals, chain-integrity verification (recomputes from genesis, reports first invalid block, expected vs actual hash/previous-hash/mekle), audit log viewer, tamper demo (clones ledger, alters a block hash, validates detection, leaves production ledger untouched), and exportable JSON via APIs.

Performance Benchmark Methodology

A synthetic population (default 1,000 voters) is generated with Argon2 passwords and Verhoeff identities. Benchmarks measure voter registration, synchronous vote submission, and batched/asynchronous submission (batches 10/20/30). Timing uses Python's monotonic time.perf_counter(). Reported: total ops, successes/fails, total duration, mean/median/p95/min/max/stdev latency, throughput. Output is saved as JSON in benchmark_results/ and displayed on the admin dashboard. Charts are rendered from actual output; no values are hardcoded.

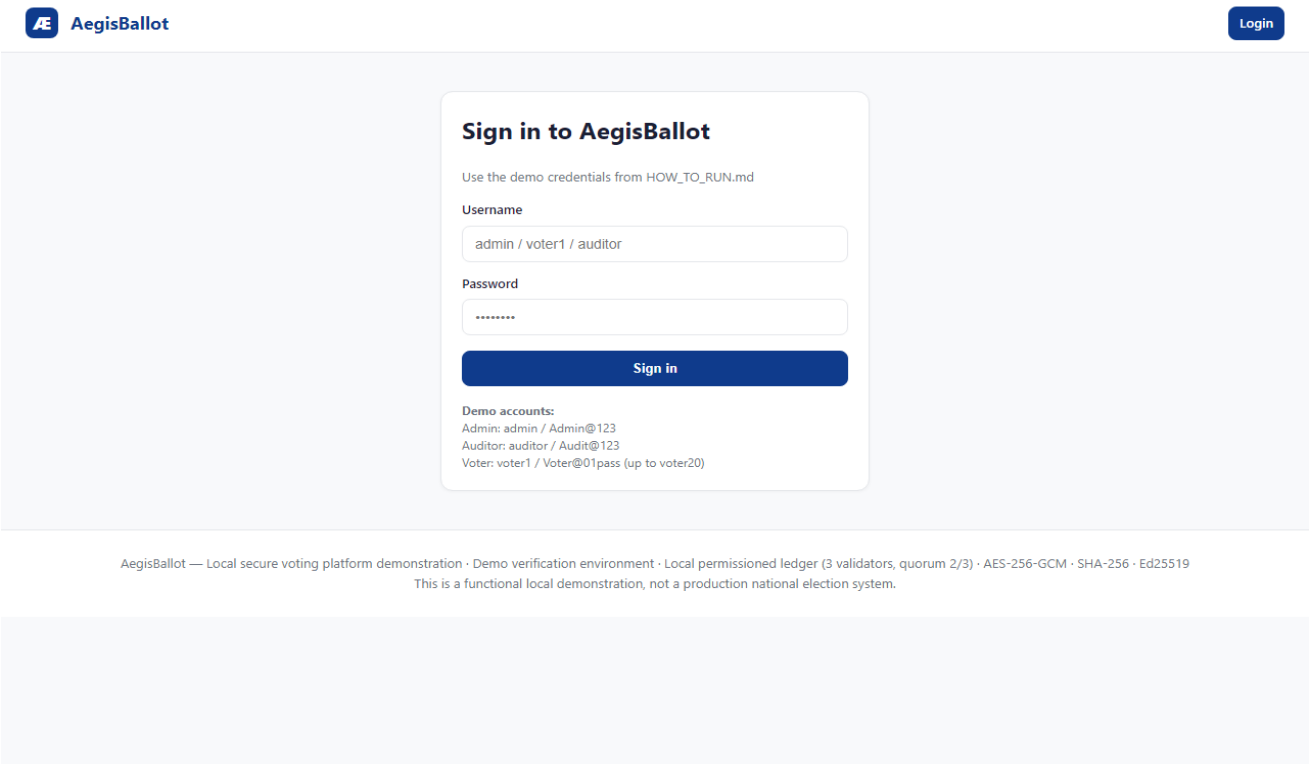
Latest measured run (2026-09-08T10:00:34.663941): 50 voters — sync throughput 62.2 ops/s, mean 16.1 ms, p95 18.0 ms; async b20 throughput 61.7 ops/s. Full JSON in benchmark_results/latest.json.

Test Coverage

Automated pytest suite covers: valid Aadhaar format, invalid checksum rejection, unknown/ineligible voter rejection, duplicate-vote rejection, unauthorized admin/auditor route rejection, tampered block detection, invalid previous-hash rejection, invalid signature handling, duplicate transaction rejection, malformed input, expired token, ballot identity separation, password hashing, audit logging, and election status enforcement. All tests assert real behavior against the running ledger and DB, not constants.

Screenshots — Real Application

The following are real screenshots captured from the running AegisBallot instance via browser automation. They reflect the light, commercial-style theme.



01_login.png

AegisBallot

Aegis Administrator ADMIN Dashboard Logout

Admin Dashboard

Monitor elections, voters, ledger health, and performance.

REGISTERED VOTERS
250

VOTES CAST
258
103.2% turnout

LEDGER HEIGHT
259
Ledger healthy

Validators

3 nodes · quorum 2/3

Node	Status	Key
validator-1	Active	13b49c247d092e8a91...
validator-2	Active	ae368d7f2bcb506e65...
validator-3	Active	acd416274a2ccfef11...

Last block: e7fca513d5cb582d... if available

Performance

Sync throughput: 57.7 ops/sec · Async (b20): 60.6 ops/sec
Mean latency: 17.32 ms · p95: 21.05 ms
View Benchmarks

Recent Elections

Manage

Title	Status	Ends
Benchmark Async 30	OPEN	2026-09-09 09:53:25.459760
Benchmark Async 20	OPEN	2026-09-09 09:53:22.153831
Benchmark Async 10	OPEN	2026-09-09 09:53:18.890320
Benchmark Async 10	OPEN	2026-09-09 09:53:05.528982
Benchmark Election	OPEN	2026-09-09 09:53:04.648242

Recent Blocks

#	Hash	Tx
258	e7fca513d5cb582d...	1
257	5711b7d14e2460a3...	1
256	0e4f66e402e6fa7b...	1
255	7e8bf4ee6998cd24...	1
254	766a56e26241ee96...	1

Chain valid: True · tip e7fca513d5cb582d...

Recent Audit Events

Time	Actor	Action	Details
2026-09-08 09:55:57.254933	admin	login_success	role ADMIN

02_admin_dashboard.png

AegisBallot

Aegis Administrator ADMIN Dashboard Logout

Election Management

Create Election

Title

Description

Start Time
dd-mm-yyyy --:--

End Time
dd-mm-yyyy --:--

Create

Benchmark Async 30 OPEN
async batch
View Pause Close Results

Benchmark Async 20 OPEN
async batch
View Pause Close Results

Benchmark Async 10 OPEN
async batch
View Pause Close Results

Benchmark Async 10 OPEN
async batch
View Pause Close Results

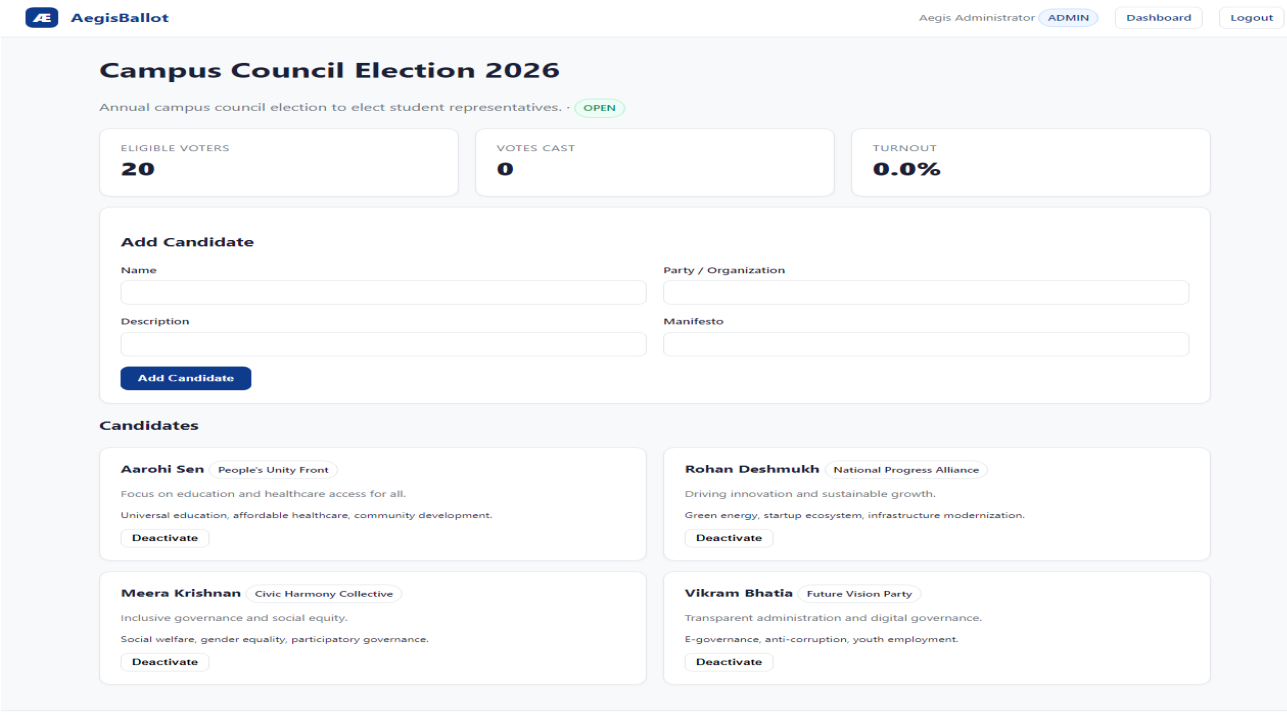
Benchmark Election OPEN
Synthetic benchmark election
View Pause Close Results

Community Welfare Board 2026 CLOSED
Election for community welfare board members.
View Open Results

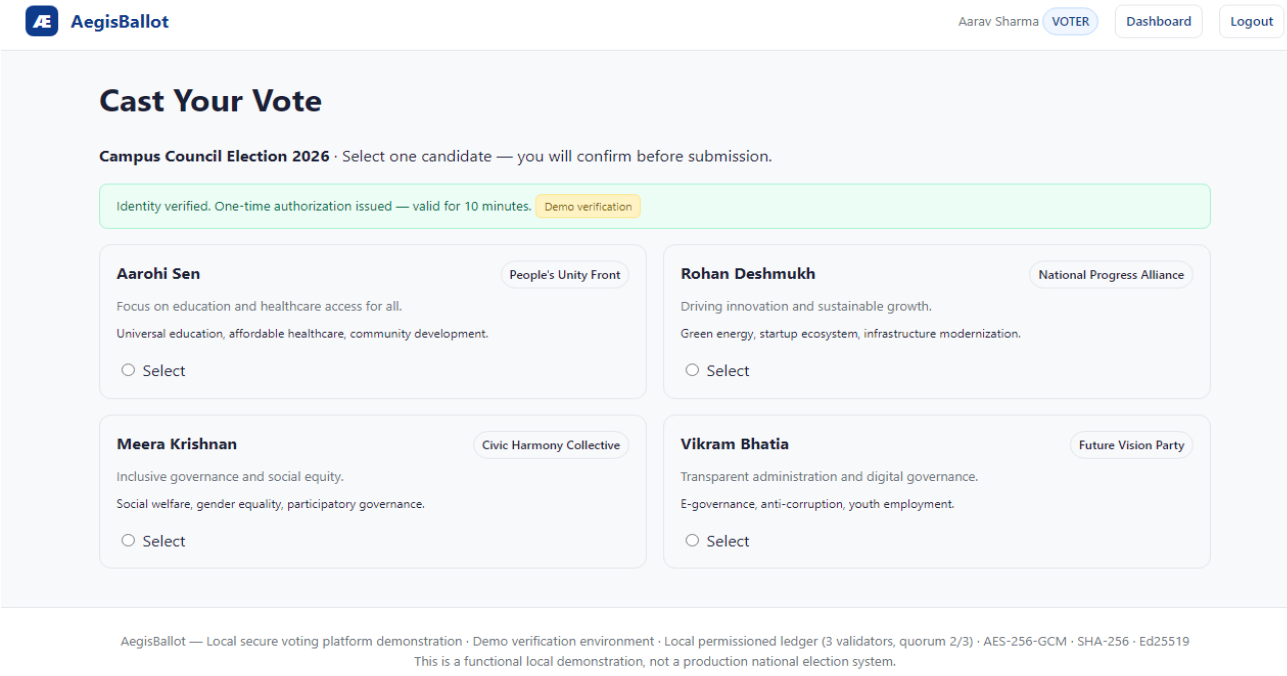
Campus Council Election 2026 OPEN
Annual campus council election to elect student representatives.
View Pause Close Results

AegisBallot — Local secure voting platform demonstration · Demo verification environment · Local permissioned ledger (3 validators, quorum 2/3) · AES-256-GCM · SHA-256 · Ed25519
This is a functional local demonstration, not a production national election system.

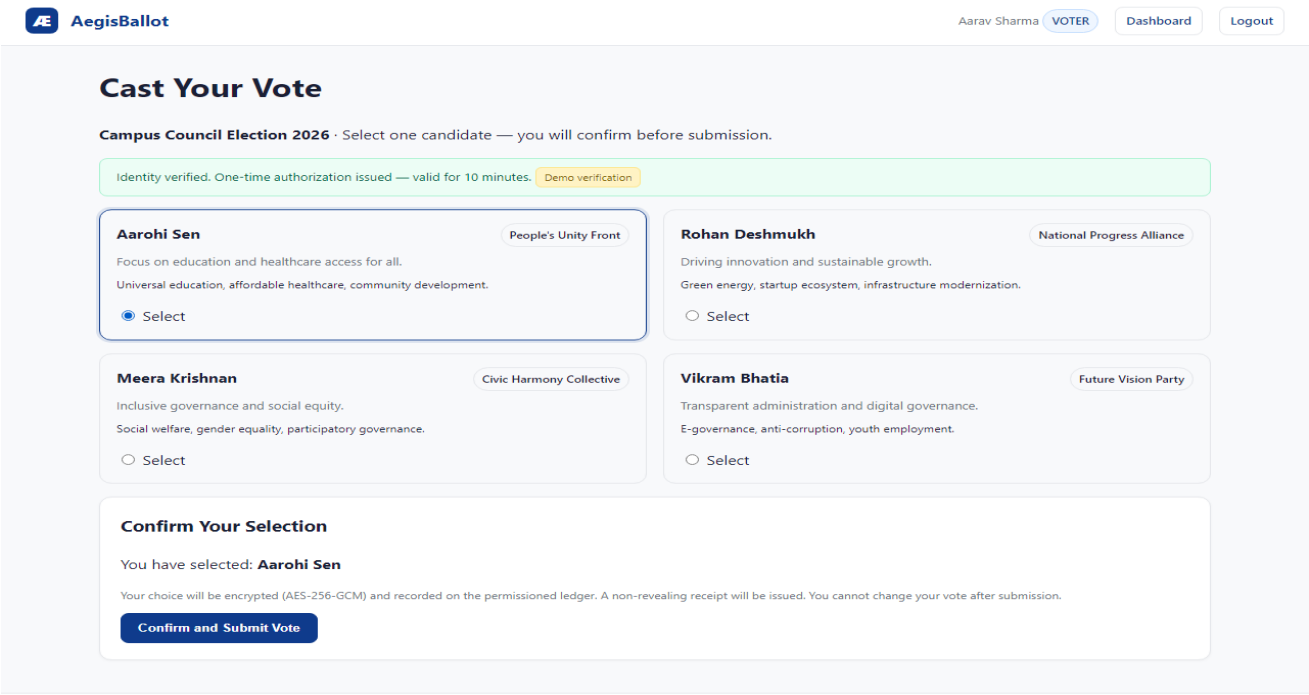
03_election_management.png



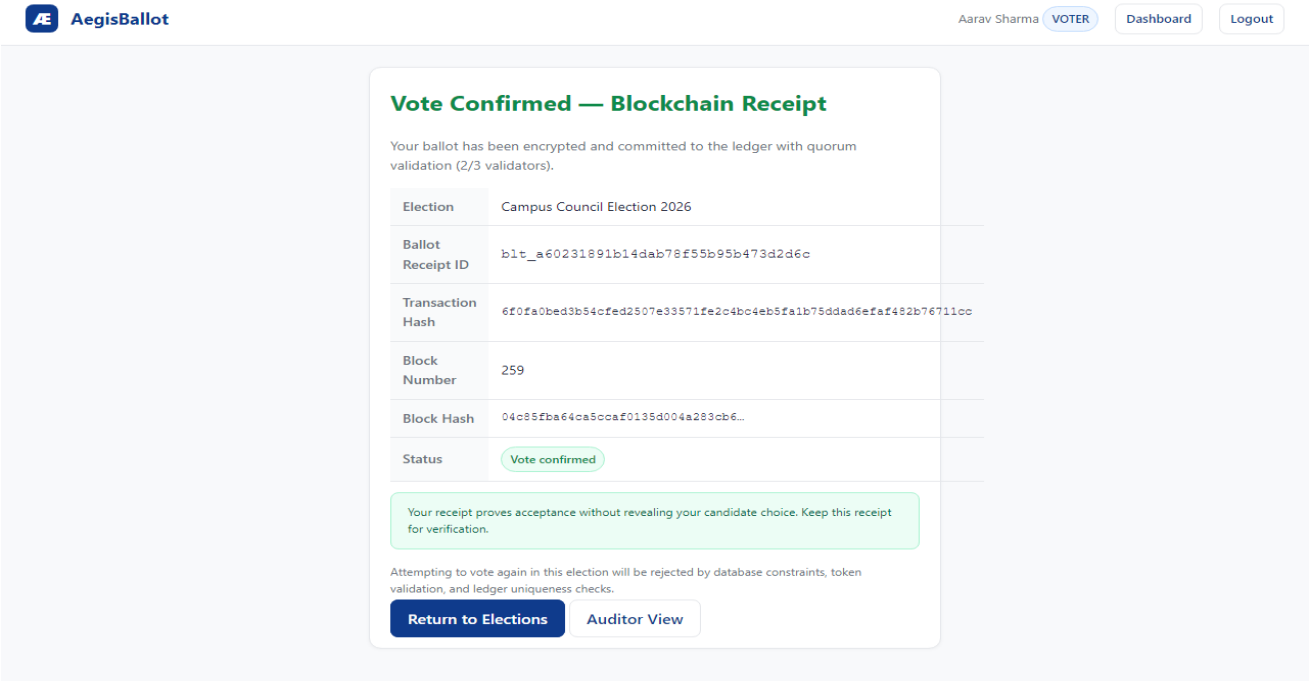
04_election_detail.png



07_candidate_selection.png



08_vote_confirmation.png



09_blockchain_receipt.png

AegisBallot

Aegis Administrator ADMIN Dashboard Logout

Results — Community Welfare Board 2026

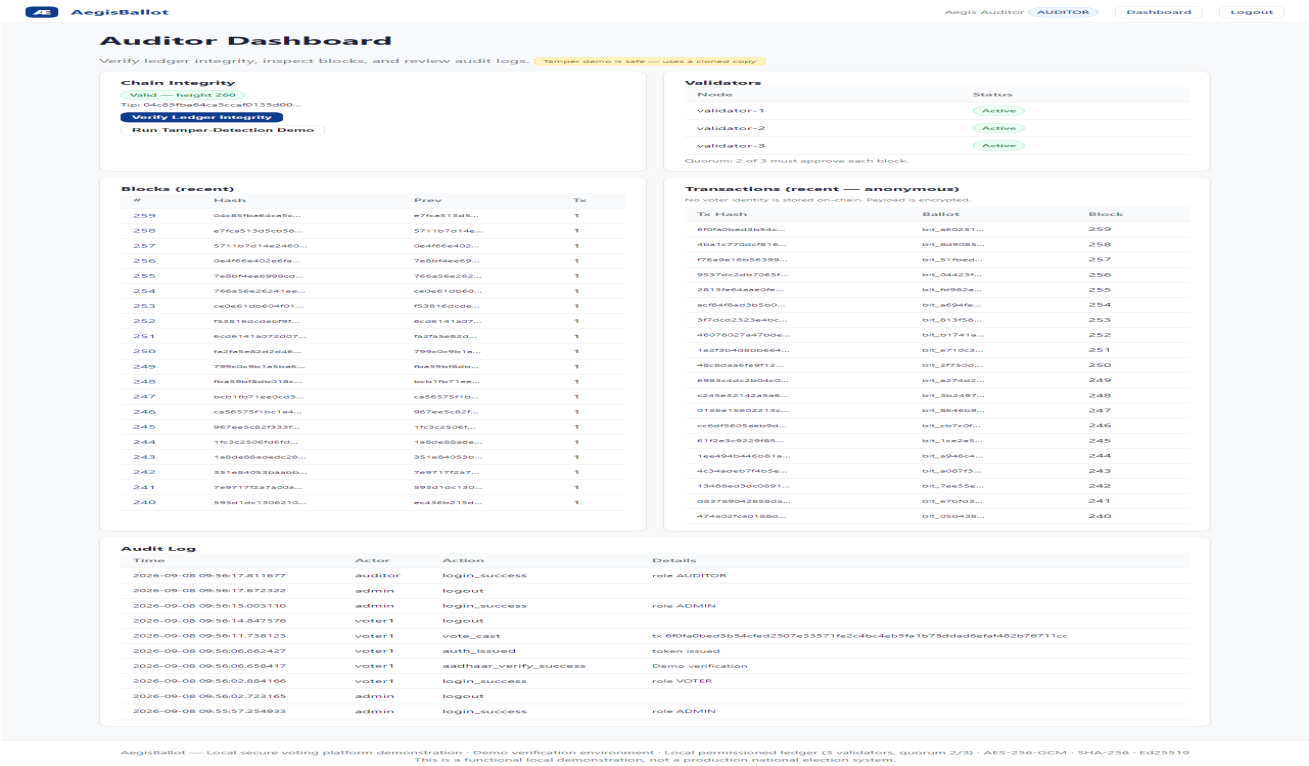
Status: CLOSED · Total votes: 8

Rank	Candidate	Party	Votes	Share
1	Aarohi Sen	People's Unity Front	**3**	37.5%
2	Rohan Deshmukh	National Progress Alliance	**3**	37.5%
3	Meera Krishnan	Civic Harmony Collective	**2**	25.0%

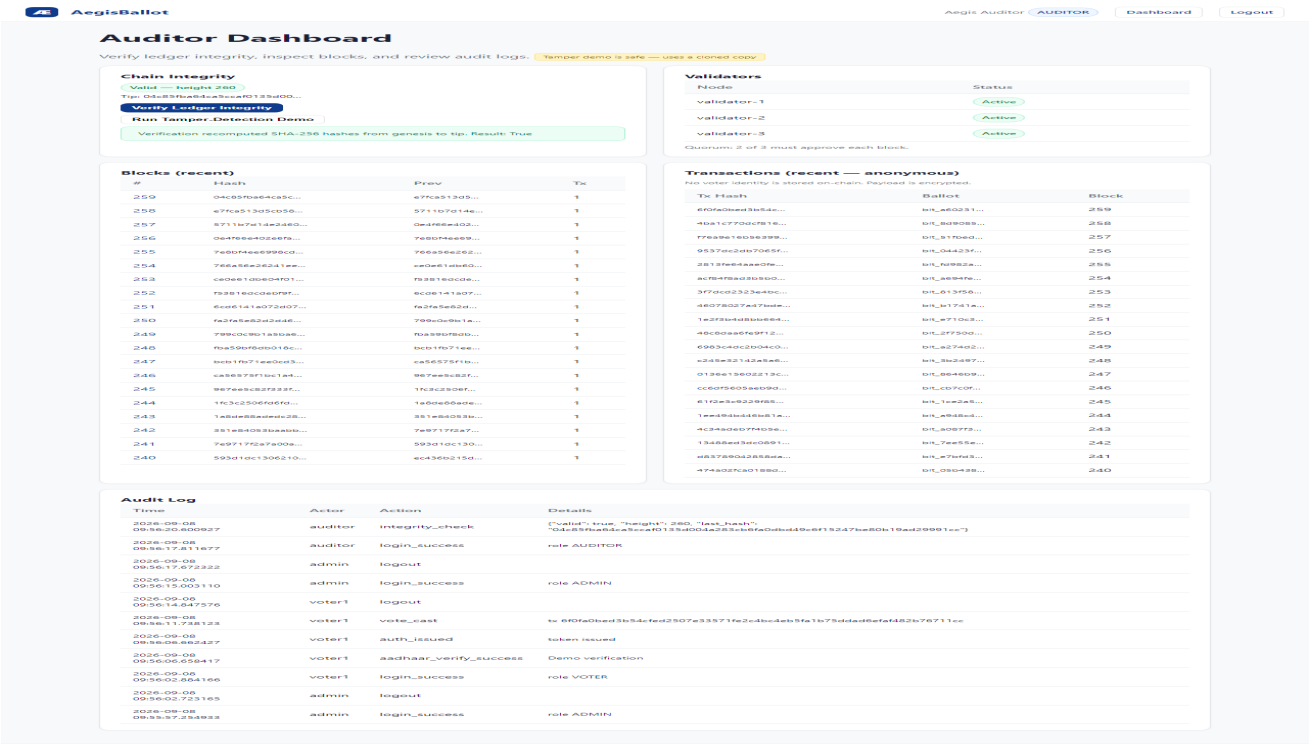
Back to Election

AegisBallot — Local secure voting platform demonstration · Demo verification environment · Local permissioned ledger (3 validators, quorum 2/3) · AES-256-GCM · SHA-256 · Ed25519
This is a functional local demonstration, not a production national election system.

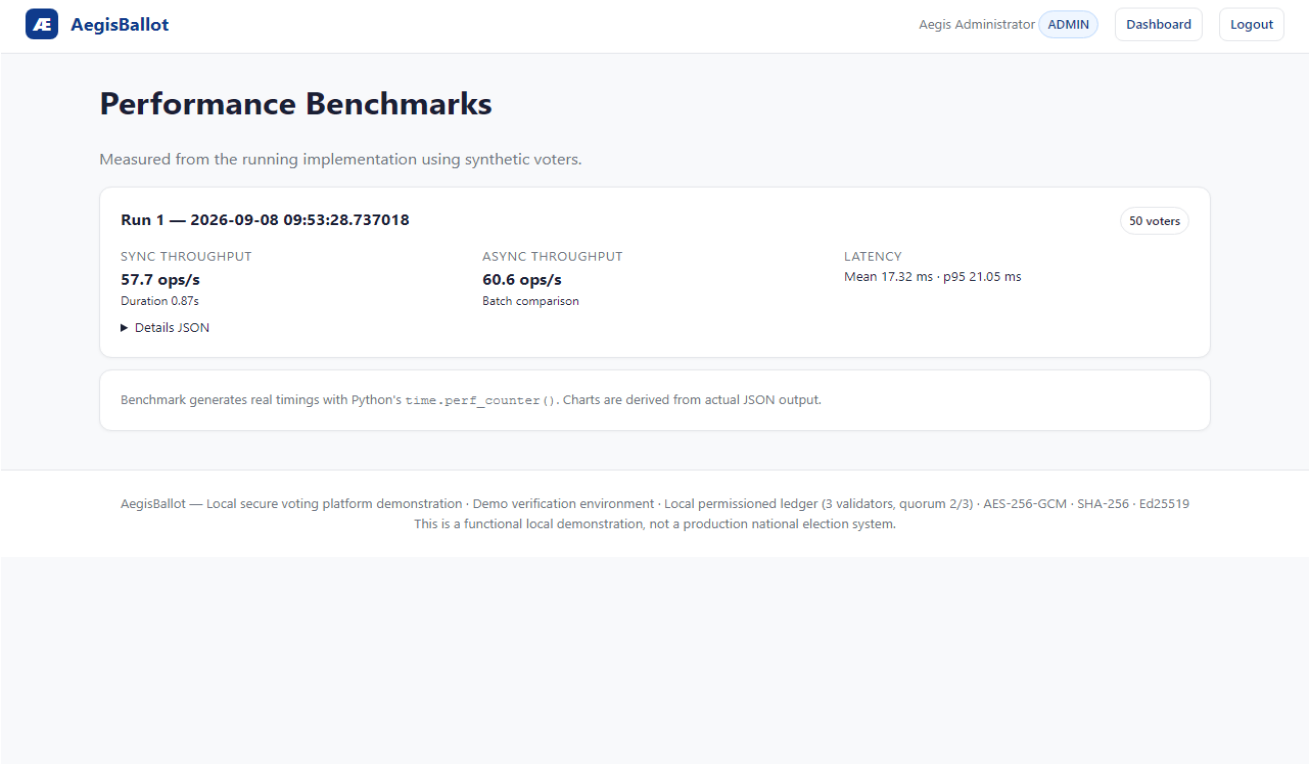
10_results.png



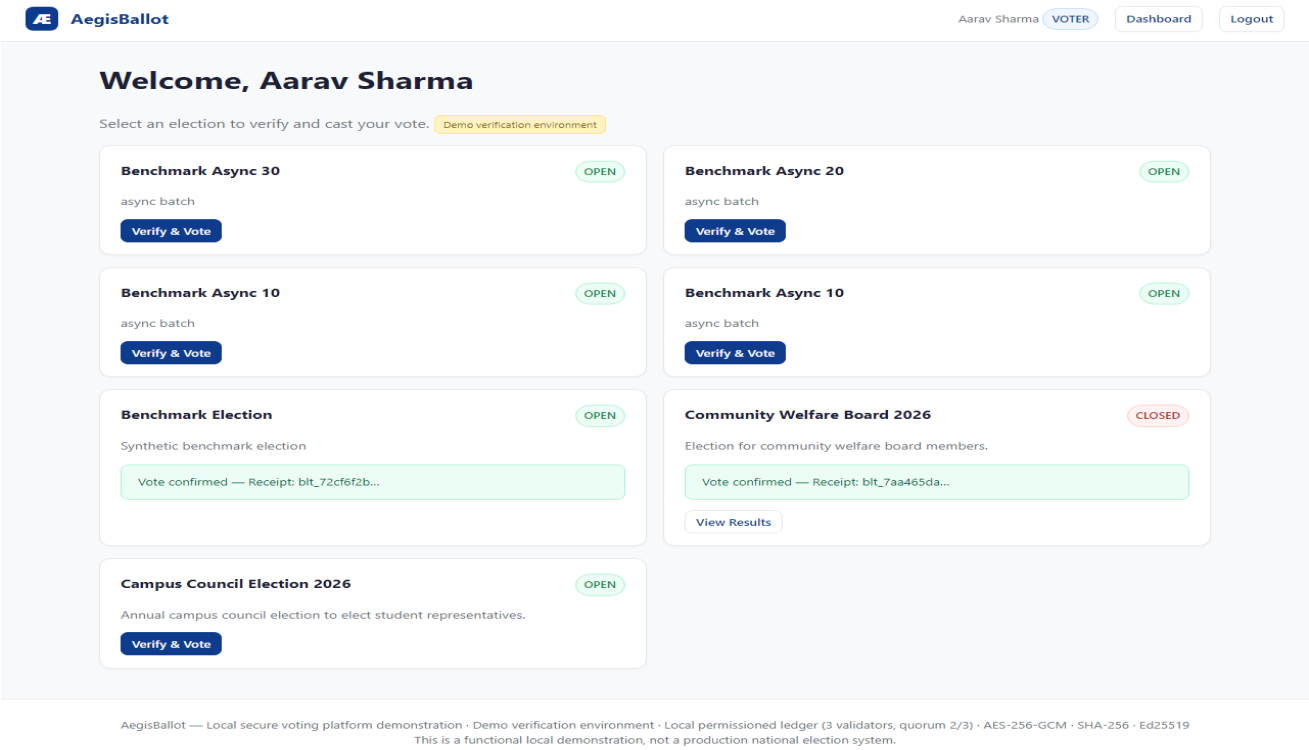
11_auditor_explorer.png



12_ledger_integrity.png



13_performance_dashboard.png



05_voter_home.png

How to Run

Prerequisites: Python 3.12 (or 3.11) and optionally Docker Desktop. No DB server or government credentials needed. Method A: double-click run.bat — creates .venv, installs, seeds, starts on 127.0.0.1:8000, opens http://localhost:8000. Method B: docker build -t aegisballot . && docker run --rm -p 127.0.0.1:8000:8000 aegisballot. Method C: python -m venv .venv, pip install -r requirements.txt, python -m app.seed, uvicorn app.main:app --host 127.0.0.1 --port 8000. See HOW_TO_RUN.md for credentials, troubleshooting, and benchmark/test commands.

Limitations and Extensibility

Local demo scope: SQLite, in-memory sessions, three local validators — not decentralized across Internet nodes. No live UIDAI, no public chain, no wallet. The ledger layer is modular: DemoAadhaarProvider can be replaced by a real identity adapter, and the ledger module can be swapped for a permissioned blockchain without changing voting flows. Future extensions: notifications, external audit export, election-key rotation, UI localization, and enhanced analytics. The system honestly describes its local guarantees and does not claim production national-election readiness.

AegisBallot — Built as a complete, polished, working local voting platform.